

Pgm4

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define CHAIRS 5
#define NUM_CUSTOMERS 3

sem_t customer;
sem_t barber;
sem_t mutex;

int waiting = 0;

void *barber_func(void *arg) {
    while (1) {
        sem_wait(&customer);
        sem_wait(&mutex);
        waiting--;
        printf("Barber cutting hair... Free chairs: %d\n", CHAIRS - waiting);
        sem_post(&barber);
        sem_post(&mutex);

        sleep(2);
        printf("Barber finished haircut.\n");
    }
}

void *customer_func(void *arg) {
    int id = *((int *)arg);
    while (1) {
        sem_wait(&mutex);
        if (waiting < CHAIRS) {
            waiting++;
            printf("Customer %d arrived. Waiting = %d\n", id, waiting);
            sem_post(&customer);
            sem_post(&mutex);
            sem_wait(&barber);
            printf("Customer %d getting haircut.\n", id);
        } else {
            printf("Customer %d leaving (no free chair).\n", id);
            sem_post(&mutex);
        }
    }
}

```

```

        sleep(rand() % 3 + 1);
    }
}

int main() {
    pthread_t barber_thread;
    pthread_t customer_threads[NUM_CUSTOMERS];
    int ids[NUM_CUSTOMERS];

    sem_init(&mutex, 0, 1);
    sem_init(&customer, 0, 0);
    sem_init(&barber, 0, 0);

    pthread_create(&barber_thread, NULL, barber_func, NULL);

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        ids[i] = i + 1;
        pthread_create(&customer_threads[i], NULL, customer_func, &ids[i]);
    }

    pthread_join(barber_thread, NULL);
    for (int i = 0; i < NUM_CUSTOMERS; i++)
        pthread_join(customer_threads[i], NULL);

    sem_destroy(&mutex);
    sem_destroy(&customer);
    sem_destroy(&barber);

    return 0;
}

```

Output:

MTECH_CSE@debianpc-02:~/Desktop/Pratham/LAB8\$./a.out

Customer 1 arrived. Waiting = 1

Customer 2 arrived. Waiting = 2

Barber cutting hair... Free chairs: 4

Customer 1 getting haircut.

Customer 3 arrived. Waiting = 2

Customer 1 arrived. Waiting = 3

Barber finished haircut.

Barber cutting hair... Free chairs: 3
Customer 2 getting haircut.
Barber finished haircut.
Customer 2 arrived. Waiting = 3
Barber cutting hair... Free chairs: 3
Customer 3 getting haircut.
Customer 3 arrived. Waiting = 3
Barber finished haircut.
Barber cutting hair... Free chairs: 3
Customer 1 getting haircut.
Barber finished haircut.
Customer 1 arrived. Waiting = 3
Barber cutting hair... Free chairs: 3
Customer 2 getting haircut.
Barber finished haircut.
Barber cutting hair... Free chairs: 4
Customer 3 getting haircut.
Customer 2 arrived. Waiting = 2
Barber finished haircut.
Barber cutting hair... Free chairs: 4
Customer 1 getting haircut.
Customer 3 arrived. Waiting = 2
Barber finished haircut.
Customer 1 arrived. Waiting = 3
Barber cutting hair... Free chairs: 3
Customer 2 getting haircut.
Customer 2 arrived. Waiting = 3
Barber finished haircut.
Barber cutting hair... Free chairs: 3
Customer 3 getting haircut.
Customer 3 arrived. Waiting = 3
Barber finished haircut.
Barber cutting hair... Free chairs: 3
Customer 1 getting haircut.
Barber finished haircut.
Customer 1 arrived. Waiting = 3
Barber cutting hair... Free chairs: 3
Customer 2 getting haircut.
Barber finished haircut.
Barber cutting hair... Free chairs: 4
Customer 3 getting haircut.
Customer 2 arrived. Waiting = 2
Barber finished haircut.
Barber cutting hair... Free chairs: 4

Customer 3 arrived. Waiting = 2

Customer 3 getting haircut.

Barber finished haircut.

Barber cutting hair... Free chairs: 4

Customer 2 getting haircut.

Customer 3 arrived. Waiting = 2

Pgm2

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int readcount = 0;      // number of active readers
sem_t mutex;           // protects readcount
sem_t wrt;             // allows writers exclusive access

void *reader(void *param) {
    while (1) {
        sem_wait(&mutex);
        readcount++;
        if (readcount == 1)
            sem_wait(&wrt);
        sem_post(&mutex);

        printf("Reader %ld is reading...\n", (long)param);
        sleep(1);

        sem_wait(&mutex);
        readcount--;
        if (readcount == 0)
            sem_post(&wrt);
        sem_post(&mutex);

        sleep(1);
    }
    return NULL;
}

void *writer(void *param) {
    while (1) {
        sem_wait(&wrt);

        printf("Writer %ld is writing...\n", (long)param);
        sleep(2);

        sem_post(&wrt);
        sleep(2);
    }
    return NULL;
}

```

```

int main() {
    pthread_t rtid[3], wtid[2];
    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);

    for (long i = 0; i < 3; i++)
        pthread_create(&rtid[i], NULL, reader, (void *)i);
    for (long i = 0; i < 2; i++)
        pthread_create(&wtid[i], NULL, writer, (void *)i);

    for (int i = 0; i < 3; i++)
        pthread_join(rtid[i], NULL);
    for (int i = 0; i < 2; i++)
        pthread_join(wtid[i], NULL);

    sem_destroy(&mutex);
    sem_destroy(&wrt);
    return 0;
}

```

Output:

MTECH_CSE@debianpc-02:~/Desktop/Pratham/LAB8\$./a.out

Reader 0 is reading...

Reader 1 is reading...

Reader 2 is reading...

Writer 0 is writing...

Writer 1 is writing...

Writer 0 is writing...

Writer 1 is writing...

Writer 0 is writing...

Writer 1 is writing...

Writer 0 is writing...

Writer 1 is writing...

Writer 0 is writing...

Pgm3

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int readcount = 0;      // number of active readers
sem_t mutex;           // protects readcount
sem_t wrt;             // allows writers exclusive access

void *reader(void *param) {
    while (1) {
        sem_wait(&mutex);
        readcount++;
        if (readcount == 1)
            sem_wait(&wrt);
        //sem_post(&mutex);
        //we'll not update the mutex. This will cause deadlock which is the requirement of the
        question.

        printf("Reader %ld is reading...\n", (long)param);
        sleep(1);

        sem_wait(&mutex);
        readcount--;
        if (readcount == 0)
            sem_post(&wrt);
        sem_post(&mutex);

        sleep(1);
    }
    return NULL;
}

void *writer(void *param) {
    while (1) {
        sem_wait(&wrt);

        printf("Writer %ld is writing...\n", (long)param);
        sleep(2);

        sem_post(&wrt);
        sleep(2);
    }
    return NULL;
}

```

```

}

int main() {
    pthread_t rtid[3], wtid[2];
    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);

    for (long i = 0; i < 3; i++)
        pthread_create(&rtid[i], NULL, reader, (void *)i);
    for (long i = 0; i < 2; i++)
        pthread_create(&wtid[i], NULL, writer, (void *)i);

    for (int i = 0; i < 3; i++)
        pthread_join(rtid[i], NULL);
    for (int i = 0; i < 2; i++)
        pthread_join(wtid[i], NULL);

    sem_destroy(&mutex);
    sem_destroy(&wrt);
    return 0;
}

```

Output:

MTECH_CSE@debianpc-02:~/Desktop/Pratham/LAB8\$./a.out

Reader 0 is reading...

//It will be stuck over here.

Pgm1

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 50

int buff[BUFFER_SIZE];
int f = 0, r = -1;
int produced_count = 0, consumed_count = 0;

sem_t mutex, full, limit;

void *producer(void *arg) {
    int i;
    for (i = 0; i < 25; i++) {
        sem_wait(&limit);
        sem_wait(&mutex);

        buff[(++r) % BUFFER_SIZE] = i;
        produced_count++;
        printf("Produced: %2d | Produced: %2d | Consumed: %2d | Diff: %2d\n",
            i, produced_count, consumed_count, produced_count - consumed_count);

        sem_post(&mutex);
        sem_post(&full);
        usleep(300000); // producer faster
    }
    return NULL;
}

void *consumer(void *arg) {
    int item, i;
    for (i = 0; i < 25; i++) {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buff[(++f) % BUFFER_SIZE];
        consumed_count++;
        printf("Consumed: %2d | Produced: %2d | Consumed: %2d | Diff: %2d\n",
            item, produced_count, consumed_count, produced_count - consumed_count);

        sem_post(&mutex);
        sem_post(&limit); // consumer frees "ahead" capacity
    }
}

```

```

        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    sem_init(&mutex, 0, 1);
    sem_init(&full, 0, 0);
    sem_init(&limit, 0, 10); // only allows producer to be max 10 ahead

    pthread_create(&thread1, NULL, producer, NULL);
    pthread_create(&thread2, NULL, consumer, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    sem_destroy(&mutex);
    sem_destroy(&full);
    sem_destroy(&limit);

    return 0;
}

```

Output:

MTECH_CSE@debianpc-02:~/Desktop/Pratham/LAB8\$./a.out

```

Produced: 0 | Produced: 1 | Consumed: 0 | Diff: 1
Consumed: 0 | Produced: 1 | Consumed: 1 | Diff: 0
Produced: 1 | Produced: 2 | Consumed: 1 | Diff: 1
Produced: 2 | Produced: 3 | Consumed: 1 | Diff: 2
Produced: 3 | Produced: 4 | Consumed: 1 | Diff: 3
Consumed: 2 | Produced: 4 | Consumed: 2 | Diff: 2
Produced: 4 | Produced: 5 | Consumed: 2 | Diff: 3
Produced: 5 | Produced: 6 | Consumed: 2 | Diff: 4
Produced: 6 | Produced: 7 | Consumed: 2 | Diff: 5
Consumed: 3 | Produced: 7 | Consumed: 3 | Diff: 4
Produced: 7 | Produced: 8 | Consumed: 3 | Diff: 5
Produced: 8 | Produced: 9 | Consumed: 3 | Diff: 6
Produced: 9 | Produced: 10 | Consumed: 3 | Diff: 7
Consumed: 4 | Produced: 10 | Consumed: 4 | Diff: 6

```

Produced: 10 | Produced: 11 | Consumed: 4 | Diff: 7
Produced: 11 | Produced: 12 | Consumed: 4 | Diff: 8
Produced: 12 | Produced: 13 | Consumed: 4 | Diff: 9
Produced: 13 | Produced: 14 | Consumed: 4 | Diff: 10
Consumed: 5 | Produced: 14 | Consumed: 5 | Diff: 9
Produced: 14 | Produced: 15 | Consumed: 5 | Diff: 10
Consumed: 6 | Produced: 15 | Consumed: 6 | Diff: 9
Produced: 15 | Produced: 16 | Consumed: 6 | Diff: 10
Consumed: 7 | Produced: 16 | Consumed: 7 | Diff: 9
Produced: 16 | Produced: 17 | Consumed: 7 | Diff: 10
Consumed: 8 | Produced: 17 | Consumed: 8 | Diff: 9
Produced: 17 | Produced: 18 | Consumed: 8 | Diff: 10